AULA 01

Como os agentes trabalham juntos

4 horas de aula
Teoria e prática

Fundamentos e arquiteturas multiagentes

Módulo 4
Sistemas Multi-Agente

Adriano Andrade

O caso que vem do Módulo 3



Uma ferramenta devolve 6 × 7 = 42.

P001 pede duas camisetas de R$ 35.

Estoque, logística e revisão dividem decisões.

A reserva conhecida continua. O trabalho novo está nas dependências entre os papéis.

A base conhecida e o próximo passo

Já construímos

Funções, mensagens e ferramentas.

Reserva identificada pelo pedido.

Limites e evidência do efeito.

Agora vamos acrescentar

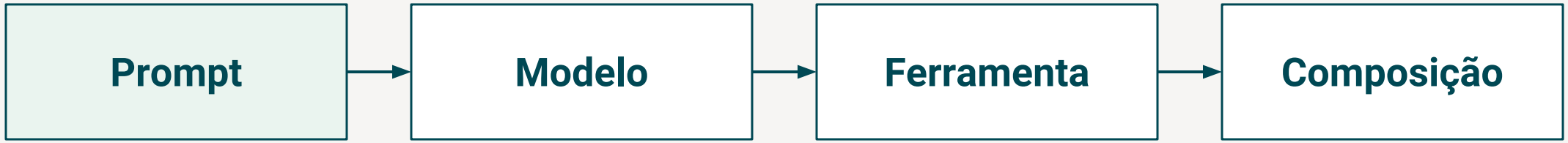
Papéis com autoridade delimitada.

Decisões que dependem de outras.

Composição com LangChain.

A mesma regra de negócio continua visível no código.

As peças do LangChain que vamos usar



Separa instruções do papel e dados do pedido.

Recebe mensagens e devolve uma resposta.

Tem nome, argumentos e uma implementação.

Liga etapas e deixa entradas e saídas visíveis.

LangChain organiza a prática desde esta aula. O núcleo usa funções e tools; o modelo real fica em caderno próprio.

Resultado de ferramenta e decisão da equipe

A ferramenta já conhecida

```
from langchain_core.tools import tool

@tool
def multiplicar(a: int, b: int) -> int:
    """Multiplica sem alterar estoque."""
    return a * b

print(multiplicar.invoke({"a": 6, "b": 7}))
```

Conta de referência

42

A chamada foi direta. Nenhum LLM escolheu a tool.

Este teste reutiliza o agente matemático conhecido do M3, agora com a interface de tool do LangChain.

A mesma ferramenta calcula o subtotal

Recorte do código

```
pedido = {"id": "P001", "item": "camiseta",
          "quantidade": 2}
preco_unitario = 35
subtotal = multiplicar.invoke({
    "a": pedido["quantidade"], "b": preco_unitario
})
print(subtotal)
```

O que observar

70

2 camisetas × R$ 35
Subtotal de P001

42 continua sendo a conta 6 × 7. O subtotal do pedido é outra chamada da mesma ferramenta.

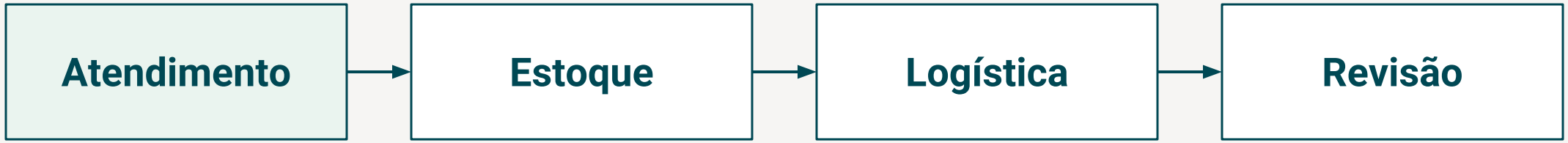
Uma conta correta encerra o pedido?

O subtotal é R$ 70. P001 já pode ser enviado?

Há duas camisetas no pedido. Ainda precisamos conferir a reserva e a etapa da logística.

Declare qual evidência falta e qual papel pode produzi-la.

Dependências no atendimento de P001



Mantém o pedido e sua intenção.

Confirma a reserva de duas camisetas.

Depende da confirmação para preparar sua etapa.

Confronta o resultado com a evidência.

Uma etapa precisa da confirmação anterior. A informação pode ficar antiga entre duas decisões.

O estoque produz uma saída para outro papel

Recorte do código

```
def conferir_estoque(dados):
    disponivel = 0 < dados["quantidade"] <= 8
    subtotal = multiplicar.invoke(
        {"a": dados["quantidade"], "b": preco_unitario})
    return {**dados, "subtotal": subtotal,
        "estoque_restante": (8 - dados["quantidade"]
                             if disponivel else 8),
        "reserva": "R-" + dados["id"] if disponivel else None,
        "status": "reservado" if disponivel else "sem_estoque"}
```

O que observar

quantidade: 2
subtotal: 70
estoque restante: 6
reserva: R-P001

Reserva e saldo são fictícios. A função reusa multiplicar; não há banco de estoque ou expedição real.

A logística depende da reserva

Recorte do código

```
def planejar_logistica(dados):
    estado = ("preparado" if dados["reserva"]
              else "sem_estoque")
    return {**dados,
        "responsavel_logistica": "Bruno",
        "status": estado}
```

O que observar

Com reserva:
status preparado

Sem reserva:
status sem_estoque

Preparado é um estado da simulação. Este papel não confirma sozinho pagamento ou envio.

A revisão confere os resultados anteriores

Recorte do código

```
def revisar(dados):
    confere = (dados["subtotal"] ==
               dados["quantidade"] * preco_unitario)
    return {**dados, "conferido": confere}
```

O que observar

70 == 2 × 35

conferido: True

É uma conferência do valor.

Conferido não autoriza pagamento ou expedição. Quantidade sem estoque também pode ter uma conta aritmética correta.

O coordenador liga as funções visíveis

Recorte do código

```
fluxo = (RunnableLambda(conferir_estoque)
         | RunnableLambda(planejar_logistica)
         | RunnableLambda(revisar))
resultado = fluxo.invoke(pedido)
```

O que observar

Entrada de estoque
→ saída para logística
→ conferência final

RunnableLambda e revisar são definidos no caderno. O operador | passa a saída de uma etapa à seguinte.

Rastro de uma rodada de P001

Etapa	Recebe	Acrescenta
Estoque	P001 e quantidade 2	Reserva R-P001
Logística	Pedido com reserva	Status preparado
Revisão	Subtotal e quantidade	Valor conferido: True/False

Leia as diferenças entre as saídas. Uma função não pode alegar ter feito a tarefa do outro papel.

Tarefas e capacidades do mesmo pedido

Tarefa	Requisito	Duração
T1 Conferir camisetas	Estoque	2
T2 Planejar coleta	Logística	3
T3 Separar embalagem	Estoque	1

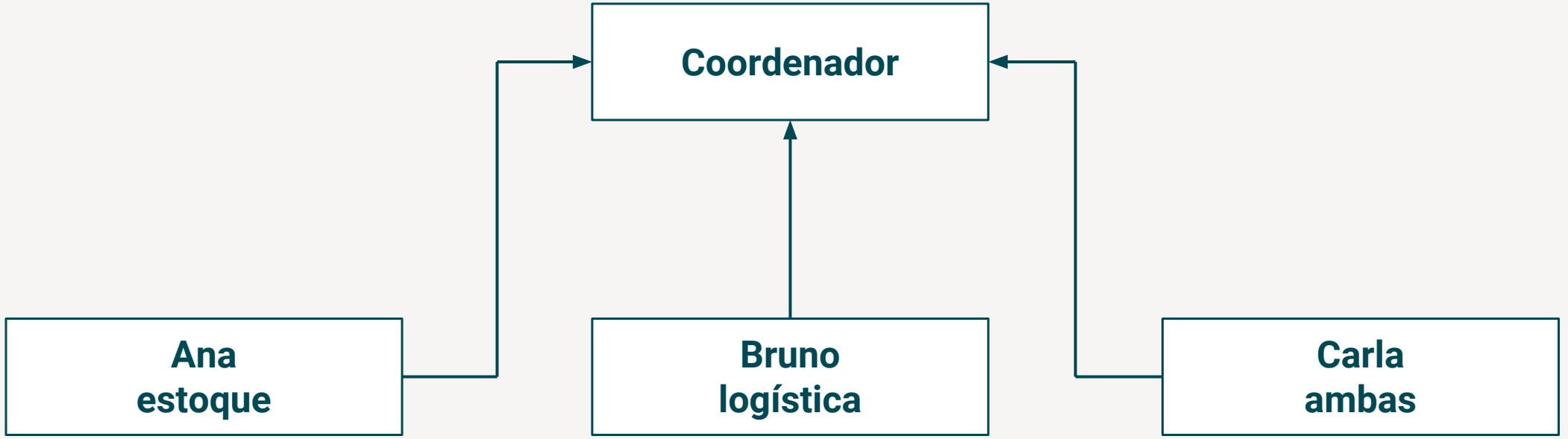
Ana: estoque. Bruno: logística. Carla: ambas. Durações em unidades didáticas.

A política dentro de um participante

Política	Decisão no exemplo
Reativa	Uma condição dispara uma ação permitida.
Deliberativa	O participante compara alternativas antes de agir.
Híbrida	Reação rápida e planejamento têm papéis distintos.

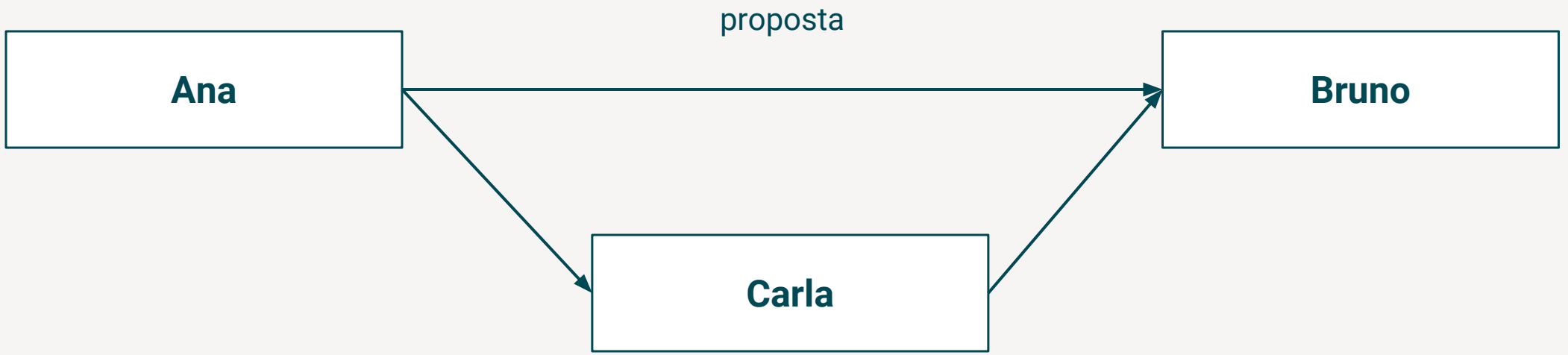
A política interna pode variar em qualquer organização da equipe. [S02]

Decisão centralizada



Os participantes informam condições. O coordenador escolhe e registra a atribuição.

Decisão entre pares



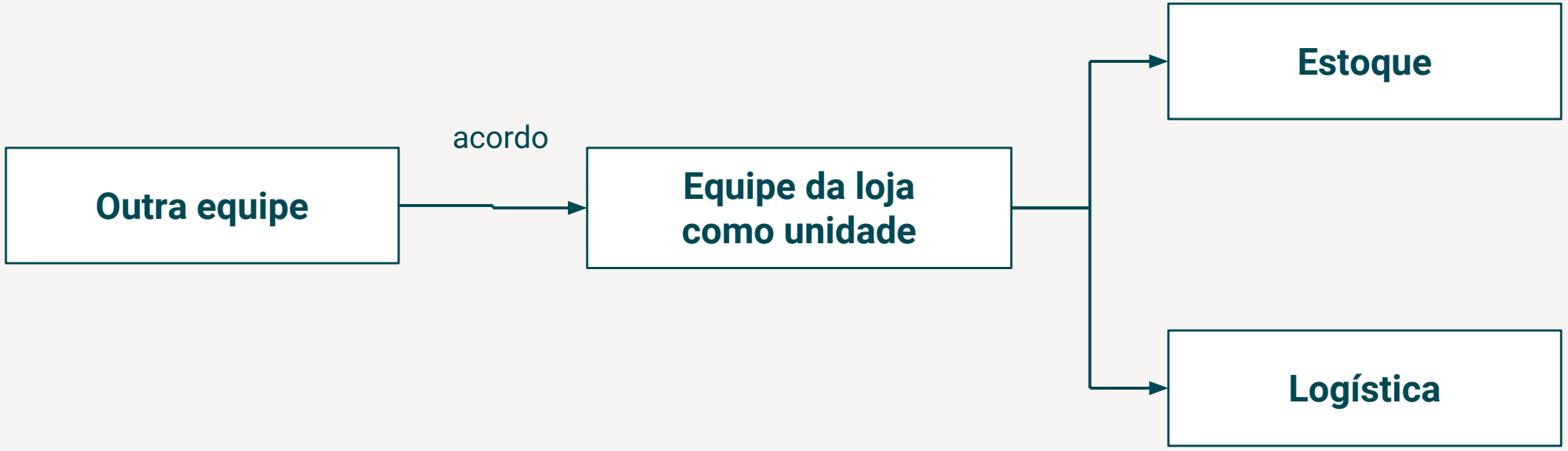
Cada participante mantém decisões locais. As mensagens precisam conciliar o uso dos recursos.

Autoridade em níveis



O atendimento coordena o pedido. Estoque e logística preservam limites locais e encaminham exceções.

Uma equipe que negocia como unidade



Um holon também integra uma unidade maior. PROSA é uma referência industrial, não uma implementação deste notebook. [S03]

Papel, autoridade e informação

Papel	Confirma	Precisa conferir
Estoque	Reserva	Item, quantidade e saldo
Logística	Sua etapa de expedição	Reserva e condições de coleta
Revisão	Parecer com evidência	Pedido e resultados anteriores

Uma pessoa ou um modelo pode exercer papéis diferentes em chamadas separadas. [S04]

Distribua as confirmações

Quem responde por cada afirmação sobre P001?

01 Associe reserva, preparação e parecer aos papéis.

02 Marque uma afirmação que precisa de evidência anterior.

03 Mostre por onde essa informação chega ao responsável.

Use uma das organizações vistas. O desenho deve mostrar autoridade e dependência.

Escolha da organização

Quem pode confirmar que P001 está pronto?

Desenhe o caminho da informação até a decisão. Indique também quem registra o efeito.

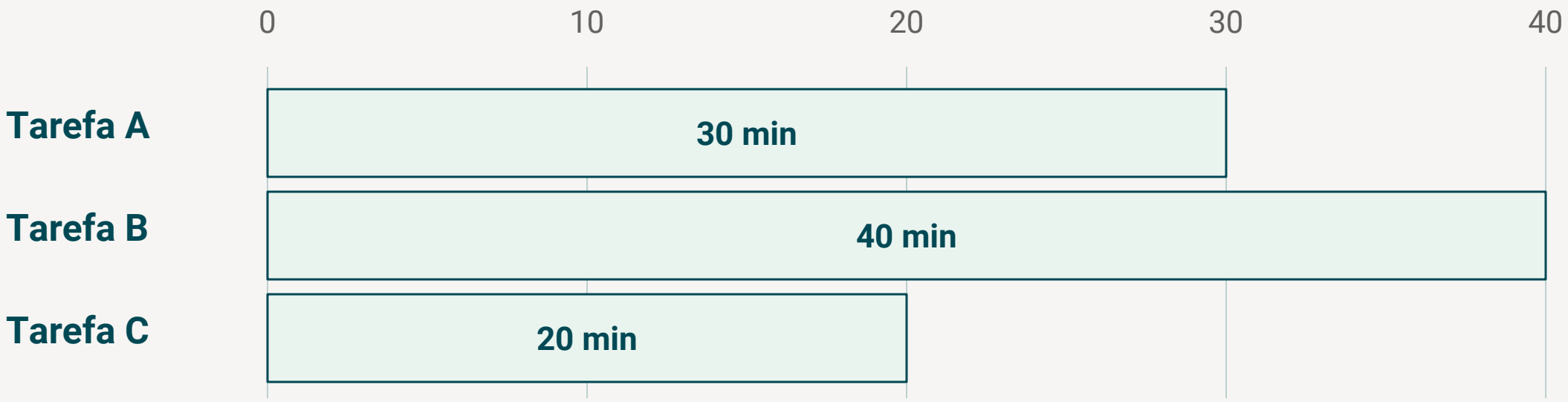
01 Autoridade central

02 Pares

03 Autoridade em níveis

Justifique sua escolha com uma dependência e uma condição de falha.

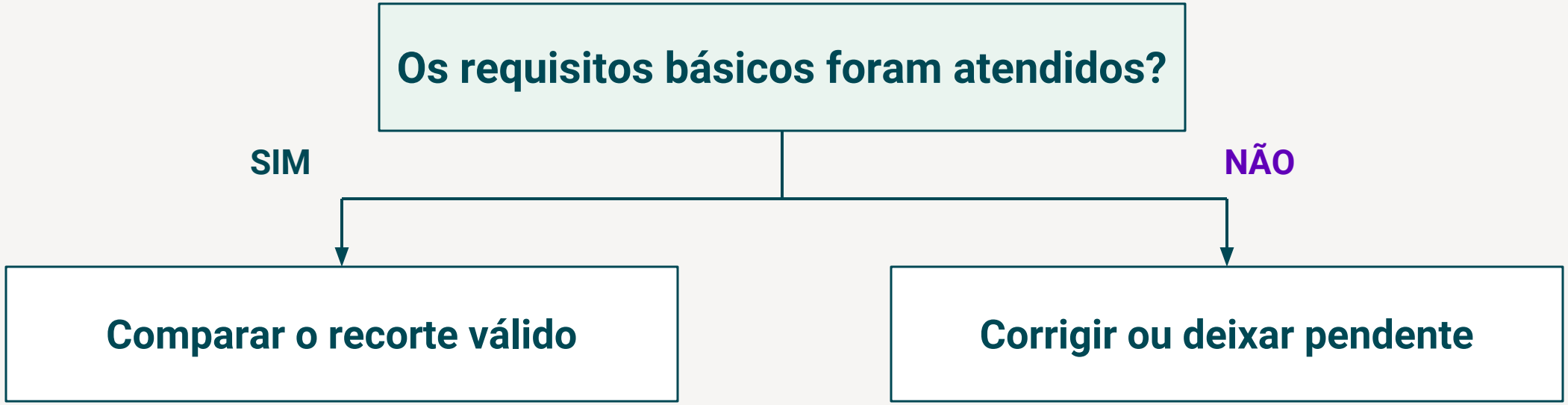
Trabalho total e tempo de conclusão



Exemplo independente do notebook: 90 min de trabalho, 40 min até concluir. Hipóteses: tarefas independentes, início zero e recursos disponíveis.

Validade da alocação

Capacidade / Atribuição única / Carga calculada



A agenda, as precedências e a indisponibilidade ainda precisam de verificações próprias.

Primeira execução com LangChain

O que entra e o que sai de cada função?

01 Leia pedido e as três funções do núcleo.

02 Preveja subtotal, reserva e status antes de invoke.

03 Execute a composição e confira o rastro.

Pratica/01_Aluno.ipynb: o algoritmo fica nas células; o framework organiza as chamadas.

Uma variação por rodada

Rodada de referência

P001: duas camisetas de R$ 35.

Subtotal previsto antes da execução.

Resultados de cada papel identificados.

Nova rodada

Mude apenas quantidade para 3.

Reexecute a partir da entrada alterada.

Confira o novo subtotal e a revisão.

A conta 6 × 7 continua 42. Mudar o pedido não altera a ferramenta nem suas permissões.

Subtotal e frete têm papéis distintos

2 × R$ 35 = R$ 70

R$ 70 + R$ 8 = R$ 78

R$ 70 é o subtotal. R$ 78 é o total quando o frete de R$ 8 é acrescentado.

Referência do caso ampliado. Não chamar o total com frete de subtotal; a conta 6 × 7 = 42 permanece separada.

Chamada direta e chamada escolhida pelo modelo

Núcleo controlado

O código escolhe a ferramenta.

RunnableLambda liga funções explícitas.

Não exige chave nem chamada remota.

Companheiro com LLM

O modelo pode propor uma tool.

A aplicação executa a chamada.

O retorno precisa ser conferido.

Uma sequência LCEL determinística não é, por si só, um agente que escolhe ações.

O agente matemático no LangChain



Pedido e regra entram em mensagens delimitadas.

ChatOllama propõe. A aplicação chama a tool.

O executor verifica resultado e permissão.

A referência original do M3 fica preservada. O novo fluxo usa LangChain desde esta aula.

Um agente LangChain com uma ferramenta

O modelo escolhe a chamada

```
agente = create_agent(
    model=llm_real, tools=[multiplicar],
    system_prompt="Use a ferramenta para a conta."
)
entrada = {"messages": [
    ("user", "Quanto é 6 vezes 7?")
]}
resposta = agente.invoke(entrada)
```

O que conferir

Pedido de ferramenta
Argumentos
Resultado 42
Resposta final

Percurso real separado: exige configuração e segredo. A execução pode falhar; não há troca silenciosa por resposta preparada.

Reprodução da decisão

Seu colega consegue reconstruir a decisão?

Entregue entrada, configuração e saída. Mostre onde a observação confirmou ou contrariou sua previsão.

01 Uma evidência

02 Uma variação

03 Uma limitação

O roteiro do professor contém as referências para discussão.

O que a Aula 2 vai receber



Subtotal, reserva e status
Papéis e evidências identificados
Uma mudança de quantidade explicada

Cada caderno básico abre sozinho. A próxima aula declara uma referência preparada; não lê automaticamente esta execução.